

Initialize a Go project:

- `go mod init <module-name>`
- `go mod init myproject`
This will:
 - Create a `go.mod` file
 - Define your module name
 - Start dependency management for your project

`go.mod`

Think of `go.mod` as the **brain of your Go project's dependency system**. It tells Go *what your project is, which dependencies it needs, and which exact versions to use*—so builds are reproducible and consistent.

1. What `go.mod` Contains

```
module github.com/tejas/myproject
go 1.22
require (
    github.com/gin-gonic/gin v1.9.0
    golang.org/x/net v0.17.0
)
```

Meaning:

- **module** → your project identity (import path)
- **go** → Go version used for compatibility rules
- **require** → list of dependencies with exact versions

2. How Go Resolves Dependencies (Internally)

`go run .`

Go does this behind the scenes:

Step 1: Read

`go.mod`

- Understand module name
- Check required dependencies

Step 2: Build Dependency Graph

- Each dependency may have its own `go.mod`
- Go recursively reads all of them
- Builds a full graph of dependencies

3. Minimal Version Selection (VERY IMPORTANT)

Go uses a concept called:

Minimal Version Selection (MVS)

How it works:

- Go picks the **lowest version** that satisfies all requirements
- Avoids unexpected upgrades

4. Role of go.sum

Purpose:

- Stores **checksums (hashes)** of dependencies
- Ensures **security & integrity**
-   Prevents:
 - Tampered packages
 - Supply chain attacks

5. Where Dependencies Are Stored

\$GOPATH/pkg/mod

Internally:

- Downloaded once
- Reused across projects
- Versioned directories

6. Commands & What They Do Internally

go mod tidy

- Removes unused dependencies
 - Adds missing ones
 - Syncs go.mod + go.sum
- go get github.com/gin-gonic/gin

Internally:

- Downloads dependency
- Updates go.mod
- Updates go.sum

go build

- Uses go.mod
- Resolves versions using MVS
- Compiles with exact versions

7. Why go.mod is Powerful

✓ Reproducible Builds

Same versions = same behavior everywhere

✓ No “dependency hell”

Unlike older systems (like vendor conflicts)

✓ Fast builds

Because of caching + minimal resolution

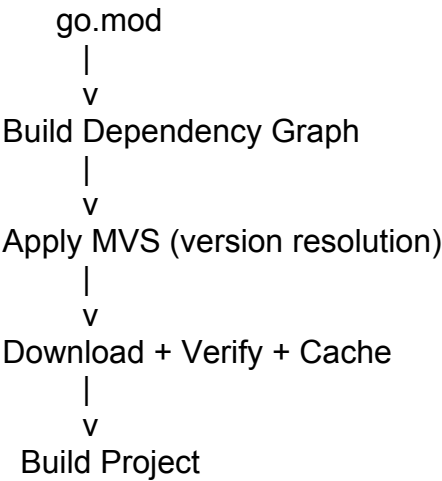
8. Advanced Concepts (Interview Level)
Indirect Dependencies

```
require github.com/pkg/errors v0.9.1 // indirect
```

Project Manifest

- └─ Who am I (module)
- └─ What I need (dependencies)
- └─ Which versions (require)
- └─ Rules (replace/exclude)

Final Mental Model



Feature	go get	go mod tidy
Purpose	Add/update dependency	Clean & fix dependencies
Updates code?	No	No
Adds deps	Yes	Yes (if used in code)
Removes deps	No	Yes (unused ones)
Use case	Install new package	Final cleanup / CI step

Go Dependencies:

go get vs go mod tidy (Internal Working)

1. Big Picture

- Read go.mod
- Resolve dependency versions (Minimal Version Selection)
- Download modules
- Verify integrity using go.sum
- Cache modules locally
- Update go.mod and go.sum

2. go get (How it works)

- Adds dependency to go.mod
- Resolves version (latest by default)
- Contacts Go module proxy (proxy.golang.org)
- Downloads module into local cache (\$GOPATH/pkg/mod)
- Verifies integrity using go.sum and checksum DB
- Updates go.mod and go.sum

3. go mod tidy (How it works)

- Scans all Go files for imports
- Adds missing dependencies
- Removes unused dependencies
- Resolves full dependency graph
- Downloads missing modules
- Cleans and syncs go.mod and go.sum

4. Key Differences

- go get: Add/update dependency
- go mod tidy: Clean and fix dependencies
- go get does not remove unused packages
- go mod tidy removes unused packages

5. Example Flow

- Import a package in code
- Run go mod tidy
- Dependency is added, downloaded, verified, and cached

6. Cache Behavior

- Dependencies stored in \$GOPATH/pkg/mod
- Reused across projects
- Works offline after first download

7. Advanced Concepts

- Direct vs indirect dependencies
- Private modules using GOPRIVATE
- Version upgrades using go get @latest

1. “Only one main.go and one main package”

✓ Correct:

- You can have **multiple files** in the same project
- But for an executable:
 - There must be **only one main package per binary**
 - Inside that package → only **one main() function**

```
myproject/  
├── main.go  
└── helper.go
```

Both can be:

```
package main
```

2. “Multiple folders allowed”

You can structure like:

```
myproject/  
├── main.go  
├── utils/  
│   └── helper.go  
└── service/  
    └── service.go
```

Each folder = **separate package**

3. “go run folderName/FileName” (Partially correct)

This works only if that file belongs to package main

```
go run cmd/app/main.go
```

```
package main
```

Important Rule:

If your code depends on multiple files in same package:

```
go run folderName/*.go
```

```
go run folderName/
```

4. “go run .”

```
go run .
```

✓ Means:

- Run current directory
- Compile all .go files in that folder (same package)

5. “Can we write multiple main methods?”

✓ Correct:

- Only one main() function per main package

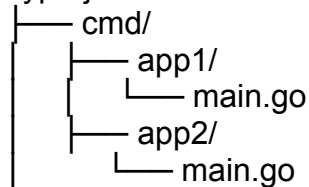
```
func main() {}
```

```
func main() {} // ERROR
```

6. Can we have multiple main in project?

Yes, but in different folders (different packages)

```
myproject/
```



✓ Each is separate executable

Run like:

```
go run cmd/app1/
```

```
go run cmd/app2/
```

7. How main() is called internally

This is important

1. You run: go run .
2. Go compiler builds binary
3. Runtime starts program
4. Looks for: package main
5. Calls: main.main()"

runtime → **init()** → **main()**

8. init() vs main()

init()

- Runs automatically
- Before main()
- Can have multiple

main()

- Entry point
- Only one allowed

Final Correction of Your Statement

Your version

only one main.go and main package

✓ Correct version:

You can have **multiple files**, but only **one main()** function per executable

Your version

go run folderName/FileName

✓ Correct version:

Works only if it's a **main package file**, otherwise run folder:

Your version

multiple main method?

✓ Correct:

Not allowed in same package

Allowed in different folders (different executables)

The Building Blocks

1. Goroutines

What they are: Functions running concurrently, managed by the Go runtime.

How they work: When you prefix a function call with go, it becomes a goroutine.

Scheduling: Goroutines are multiplexed onto threads by the Go scheduler. You don't control threads directly; the runtime does.

Scalability: You can easily spin up thousands of goroutines because they start with a very small stack (a few KB) that grows dynamically.

```
func sayHello() {  
    fmt.Println("Hello from goroutine")  
}  
func main() {  
    go sayHello()  
    fmt.Println("Hello from main")  
}
```

Here, both messages may appear in different orders depending on scheduling.

CODE 1

```
package main
import "fmt"
func display(str string) {
    for i := 0; i < 3; i++ {
        fmt.Println(str)
    }
}

func main() {
    go display("Hello, Goroutine!")// Runs
    concurrently
    display("Hello, Main!")
}
```

CODE 2

```
package main
import (
    "fmt"
    "time")
func display(str string) {
    for i := 0; i < 3; i++ {
        fmt.Println(str)
    }
}

func main() {
    go display("Hello, Goroutine!")
    time.Sleep(time.Second)// Wait for goroutine
    display("Hello, Main!")
}
```

Aspect	Code 1  (No Sleep)	Code 2  (With Sleep)
Goroutine Start	Starts but not guaranteed to run	Starts and gets time to run
Main Execution	Runs immediately	Pauses for 1 second
Synchronization	 No synchronization	 Temporary (Sleep-based)
Program Exit	Exits quickly	Delayed exit
Goroutine Completion	 May NOT complete	 Likely completes
Output Reliability	 Unpredictable	 Mostly predictable
Output Order	Random / missing goroutine output	Usually goroutine first
Scheduler Role	May not schedule goroutine in time	Gets time to schedule properly
Performance Control	No control	Artificial delay
Real-world Usage	 Not usable	 Not recommended
Best Practice	 Wrong approach	 Temporary fix
Risk	Goroutine gets killed	Wrong timing assumption
Blocking Behavior	No blocking	Main is blocked for fixed time

2. Channels

Purpose: Safe communication between goroutines.

Mechanics: Channels are typed conduits; you can send and receive values of a specific type.

Synchronization: Sending blocks until another goroutine receives (for unbuffered channels).

Buffered channels: Allow a fixed number of values to be queued without blocking.

```
ch := make(chan string)
go func() {
    ch <- "message from goroutine"}()
msg := <-chfmt.Println(msg)
```

In Go language, a channel is a medium through which a goroutine communicates with another goroutine and this communication is lock-free. Or in other words, a channel is a technique which allows to let one goroutine to send data to another goroutine. By default channel is bidirectional, means the goroutines can send or receive data through the same channel

3. Select Statement

Why it matters: Lets you wait on multiple channel operations simultaneously.

Behavior: Executes the first case that's ready. If multiple are ready, it picks one at random.

Use case: Timeouts, multiplexing input, or coordinating multiple goroutines.

```
select {case msg := <-ch1:
    fmt.Println("Received:", msg)case msg := <-ch2:
    fmt.Println("Received:", msg)case <-time.After(2 * time.Second):
    fmt.Println("Timeout")}
```

How select Works Internally

- Go checks all cases
- If **one is ready** → execute it
- If **multiple are ready** → pick randomly
- If **none are ready** → block (wait)

Under the Hood

Go Scheduler: Uses an M:N model (M goroutines mapped to N OS threads). It balances goroutines across available CPU cores.

Work Stealing: Goroutines are distributed across processors, and idle processors can “steal” work from busy ones.

Preemption: The scheduler can pause long-running goroutines to ensure fairness.

Concurrency Safety

Race Conditions: Occur when multiple goroutines access shared memory without synchronization.

Best Practice: Prefer channels for communication instead of shared memory. If you must share memory, use sync primitives like sync.Mutex.

Concurrency Patterns in Go

Worker Pools: Efficiently process jobs with a fixed number of goroutines.

Fan-out / Fan-in: Spread work across multiple goroutines (fan-out) and then collect results (fan-in).

Pipeline: Chain stages of processing, each stage running in its own goroutine and connected by channels.

Philosophy Behind Go's Concurrency

Go's model is inspired by **Communicating Sequential Processes (CSP)**. The mantra is:

"Do not communicate by sharing memory; instead, share memory by communicating."

This means goroutines should interact through channels rather than directly manipulating shared state.

In summary: Concurrency in Go is about goroutines (lightweight concurrent tasks), channels (safe communication), and the select statement (coordination). The runtime scheduler makes it efficient, and the CSP philosophy keeps it safe and structured.

*** Database Connection ***

First Install the Mysql Driver : [go get github.com/go-sql-driver/mysql](https://github.com/go-sql-driver/mysql)

```
package main
```

```
// Defines the package
```

```
// main is required to create an executable program
```

```
import (
```

```
"database/sql"
```

```
"fmt"
```

```
"log"
```

```
_ "github.com/go-sql-driver/mysql"
```

```
)
```

Import section:

- database/sql → Go's standard package to work with databases
- fmt → for printing output
- log → for logging errors

```
// _ "github.com/go-sql-driver/mysql"
```

- Used only to register MySQL driver internally

- You don't call it directly

```
func main() {
```

```
// Entry point of program
```

```
// Execution starts from here
```

```
dsn := "root:Root@1234@tcp(localhost:3306)/db"
```

```
// DSN = Data Source Name (connection string)
```

```
// root      → username
```

```
// Root@1234  → password
```

```
// tcp(...)   → connection type
```

```
// localhost:3306 → host + port
```

```
// db         → database name
```

```

db, err := sql.Open("mysql", dsn)
//Opens a connection handle (not actual connection yet)
• "mysql" → driver name
• dsn → connection string
• sql.Open does NOT connect immediately

if err != nil {
fmt.Println("Connection opened successfully!", err) // Don't write
log.Fatal("Error opening connection:", err) // Only write log.fatal or Panic
panic(err) // After write a log.fatal panic never runs
}
defer db.Close()
// Ensures DB connection closes when program ends
// defer = run this at the end

// Check connection
err = db.Ping()
// Actually checks real connection to DB

if err != nil {
log.Fatal("Error connecting to DB:", err)
}

fmt.Println("Connected to database!")
// Confirmation message

// Query
rows, err := db.Query("SELECT roll_no, name FROM student")
if err != nil {
fmt.Println("Error: ", err)
log.Fatal("Query error:", err)
}
defer rows.Close()
// Closes result set after use

// Iterate rows
for rows.Next() {
var rollNo int
var name string

err := rows.Scan(&rollNo, &name)
// Reads data from row into variables
// & → address (pointer)

if err != nil {
log.Fatal(err) }

fmt.Println(rollNo, name) }

```

```
// Check for errors after loop
err = rows.Err()
if err != nil {
log.Fatal(err) }}
```

Start



Open DB (sql.Open) // Creates connection pool, NOT actual connection



Ping DB (db.ping) // Actually connects to database



Execute Query



Loop rows



Scan data



Print data



Close rows



Close DB

***** REST API *****

```
package main
```

```
import (
```

```
"encoding/json" // This package is used to convert data between Go and JSON format.
```

```
"fmt"           // Used for printing output to console (terminal).
```

```
"net/http"      // main package to build web servers in Go.
```

- HTTP server
- Request handling
- Response writing
- Routing

```
"strconv"      // Used for converting data types, mainly: String ↔ Integer
```

```
)
```

```
// Struct (Model)
```

```
type User struct {
```

```
ID int `json:"id"`
```

```
Name string `json:"name"`
```

```
}
```

This defines a **structure (like a class in Java)** to represent a user. It has two fields: ID and Name. The part `json:"id"` tells Go how to convert this struct into JSON format when sending responses.

For example, ID will be shown as "id" in API output instead of "ID". This struct is used everywhere in your program to store and transfer user data.

```
// Dummy Data (In-memory)
```

```
var users = []User{
```

```
{ID: 1, Name: "Tejas"},
```

```
{ID: 2, Name: "Rahul"},
```

```
}
```

Here you are creating a **slice (dynamic array)** of users. This slice is acting like a **temporary database** stored in RAM. When your API runs, all operations (add, update, delete) happen on this slice. But once you stop the server, this data disappears because it is not saved anywhere. So this is good for practice but not for real-world applications.

```
// GET: Get All Users
```

```
// POSTMAN APP : http://localhost:8080/users
```

```
func getUsers(w http.ResponseWriter, r *http.Request) {
```

- getUsers → function name
- w → used to send response
- r → contains request data

```
w.Header().Set("Content-Type", "application/json") // I am sending data in JSON format"
```

```
json.NewEncoder(w).Encode(users)
```

```
// Convert Go → JSON
```

```
// Send response
```

- w is like a **pipe to client**
- Data is written into it
- Client receives it

```
}
```

This function is used to **return all users**. The w parameter is used to send response back to the client, and r contains request details. First, you set the response type as JSON so the client understands the format. Then you convert the users slice into JSON and send it back. So whenever someone hits /users with GET method, they receive the full list of users.

```
// GET: Get User by ID
```

```
// POSTMAN APP : http://localhost:8080/users?id=1
```

```
func getUserByID(w http.ResponseWriter, r *http.Request) {
```

```
idParam := r.URL.Query().Get("id")
```

This line extracts the id from the URL like /users?id=1. Since URL values are always strings, you convert it into integer using strconv.Atoi. Then you loop through the users slice to find the matching ID. If a match is found, that user is returned as JSON. If no match is found, an error message "User not found" is sent.

```
// Convert string → int
```

```
id, err := strconv.Atoi(idParam)
```

```
if err != nil {
```

```
http.Error(w, "Invalid ID", http.StatusBadRequest)
```

```
return
```

```
}
```

```
// Search user
for _, user := range users {
if user.ID == id {
w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(user)
return // Stops function execution immediately & Prevents unnecessary looping
}
}
// If not found
http.Error(w, "User not found", http.StatusNotFound)
}
```

```
// POST: Create User
// POSTMAN APP : http://localhost:8080/users
func createUser(w http.ResponseWriter, r *http.Request) {
var newUser User
err := json.NewDecoder(r.Body).Decode(&newUser)
```

This function reads the **request body (JSON input)** and converts it into a Go struct.

For example, if user sends { "id": 3, "name": "Amit" }, it becomes a User object. Then you append this new user into the users slice using append(). Finally, you return the newly created user as confirmation. This is how data is added in your API.

```
// json.NewDecoder(r.Body) => It prepares to read JSON data from request body
//.Decode(&newUser) =>
```

What it does:

- Reads JSON from r.Body
- Converts it into Go struct
- Stores it inside newUser

Why &newUser ?

- & means **address of variable**
- Go needs address to **fill data inside it**

```

if err != nil {
    http.Error(w, err.Error(), http.StatusBadRequest)
    return
}
users = append(users, newUser)
w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(newUser)
}

// PUT: Update User
// POSTMAN APP : http://localhost:8080/users?id=1
func updateUser(w http.ResponseWriter, r *http.Request) {
    idParam := r.URL.Query().Get("id")
    id, err := strconv.Atoi(idParam)
    if err != nil {
        http.Error(w, "Invalid ID", http.StatusBadRequest)
        return
    }
    var updatedUser User
    err = json.NewDecoder(r.Body).Decode(&updatedUser)
    if err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }
    for i, user := range users {
        if user.ID == id {
            users[i].Name = updatedUser.Name
            json.NewEncoder(w).Encode(users[i])
            return
        }
    }
    http.Error(w, "User not found", http.StatusNotFound)
}

```



```
// DELETE: Remove User
// POSTMAN APP : http://localhost:8080/users?id=1
func deleteUser(w http.ResponseWriter, r *http.Request) {
    idParam := r.URL.Query().Get("id")
    id, err := strconv.Atoi(idParam)
    if err != nil {
        http.Error(w, "Invalid ID", http.StatusBadRequest)
        return
    }
    for i, user := range users {
        if user.ID == id {
            users = append(users[:i], users[i+1:]...) // Important: Delete user from slice
            // This is used to remove a user from slice. It works by combining:
            

- elements before index
- elements after index


            // So the selected user is removed from memory. After deletion, a message "User deleted" is returned.
            // If user is not found, an error is sent.
            w.Write([]byte("User deleted"))
            return
        }
    }
    http.Error(w, "User not found", http.StatusNotFound)
}
```

```
func main() {
    http.HandleFunc("/users", func(w http.ResponseWriter, r *http.Request) {
```

This line tells Go:

“Whenever a request comes to /users, run this function.”

Inside this function, you decide what action to take based on the request type. This is called **routing**. It connects your URL with your logic. Every request to /users goes through this block.

```
switch r.Method {
```

This checks **what type of request client sent** (GET, POST, PUT, DELETE). Based on that, it calls the correct function. For example, GET calls getUsers, POST calls createUser, etc. This is important

because the same URL /users is used for multiple operations. So method decides what action to perform.

```
case http.MethodGet:
id := r.URL.Query().Get("id")
if id != "" {
getUserByID(w, r)
return // VERY IMPORTANT
}
getUsers(w, r)
case http.MethodPost:
createUser(w, r)
case http.MethodPut:
updateUser(w, r)
case http.MethodDelete:
deleteUser(w, r)
default:
http.Error(w, "Method Not Allowed", http.StatusMethodNotAllowed)
}
})
```

```
fmt.Println("Server running on port 8080...")
http.ListenAndServe(":8080", nil)
```

This line starts your server on port 8080. It listens for incoming requests continuously. Whenever a request comes, it matches the route and executes your logic. The nil means you are using the default router provided by Go. Without this line, your API will not run.

```
}
```